

UART Receiver Design & Verification

Assignment 7 — RTL Implementation, Lint Audit & ModelSim Verification

Student Name: Ziad Mostafa Abdelaziz
Course: Digital IC Design Course
Simulator: ModelSim Intel FPGA Edition v10.5b
Lint Tool: SpyGlass vL-2016.06
Baud Rate: 115.2 KHz (Reference TX Clock)
Date: August 2026

Contents

1	Design Overview	2
1.1	Frame Structures	2
1.2	Block Interface	2
1.3	Oversampling and 3-Sample Majority Voting	2
2	System Architecture	3
3	RTL Source Code	3
3.1	Top Module — <code>UART_RX.v</code>	3
3.2	FSM Controller — <code>uart_rx_fsm.v</code>	5
3.3	Data Sampling Module — <code>data_sampling.v</code>	7
3.4	Edge and Bit Counter — <code>edge_bit_counter.v</code>	8
3.5	Deserializer Module — <code>deserializer.v</code>	8
3.6	Start Bit Checker — <code>strt_check.v</code>	9
3.7	Parity Checker — <code>parity_check.v</code>	9
3.8	Stop Bit Checker — <code>stp_check.v</code>	9
4	SpyGlass Lint Audit	10
4.1	<code>moresimple.rpt</code> Summary	10
4.2	Rule Analysis	10
5	Simulation & Verification Results	11
5.1	ModelSim Transcripts	12
5.2	Waveform Timing Analysis	13
6	FSM & Design Quality Discussion	13
7	Conclusion	13

Design Overview

A Universal Asynchronous Receiver/Transmitter (UART) is a core serial communication module. In Full Duplex mode, the Receiving UART converts incoming serial data into an 8-bit parallel byte for master devices.

The UART Receiver (UART_RX) module is designed with an oversampling architecture to reliably sample incoming data at configurable prescale rates ($8\times$, $16\times$, or $32\times$) relative to the transmit clock ($f_{TX} = 115.2$ KHz).

Frame Structures

The design supports three frame configurations controlled by PAR_EN (Parity Enable) and PAR_TYP (Parity Type):

1. **Even Parity Mode (PAR_EN = 1, PAR_TYP = 0):**

START (0) | b0 b1 b2 b3 b4 b5 b6 b7 (LSB first) | EP | STOP (1)

Where $EP = \bigoplus P_DATA$ (XOR of all 8 data bits).

2. **Odd Parity Mode (PAR_EN = 1, PAR_TYP = 1):**

START (0) | b0 b1 b2 b3 b4 b5 b6 b7 (LSB first) | OP | STOP (1)

Where $OP = \overline{\bigoplus P_DATA}$ (XNOR of all 8 data bits).

3. **No Parity Mode (PAR_EN = 0):**

START (0) | b0 b1 b2 b3 b4 b5 b6 b7 (LSB first) | STOP (1)

Block Interface

Table 1: UART RX Interface Signal Description

Port Name	Width	Direction	Description
CLK	1	Input	Receiver Clock Signal ($f_{RX} = \text{Prescale} \times 115.2$ KHz)
RST	1	Input	Synchronized active-low reset signal
PAR_TYP	1	Input	Parity Type Select (0: Even, 1: Odd)
PAR_EN	1	Input	Parity Enable (1: Enabled, 0: Disabled)
Prescale	6	Input	Oversampling ratio (8, 16, or 32)
RX_IN	1	Input	Serial input bit stream
P_DATA	8	Output	Parallel output byte
data_valid	1	Output	1-clock cycle high pulse on valid frame
Parity_Error	1	Output	High on parity check failure
Stop_Error	1	Output	High on stop bit framing failure

Oversampling and 3-Sample Majority Voting

To filter out line noise and glitches, UART_RX uses a 3-sample majority voter at the center of every bit period.

Given a prescale factor N , the samples are taken at clock ticks:

$$T_1 = \frac{N}{2} - 1, \quad T_2 = \frac{N}{2}, \quad T_3 = \frac{N}{2} + 1$$

Table 2: Center Sampling Ticks per Prescale Value

Prescale (N)	RX Clock Frequency	Sample Ticks (T_1, T_2, T_3)
8	921.6 KHz	3, 4, 5
16	1.8432 MHz	7, 8, 9
32	3.6864 MHz	15, 16, 17

The majority voted output bit is computed as:

$$\text{sampled_bit} = (S_1 \cdot S_2) \vee (S_1 \cdot S_3) \vee (S_2 \cdot S_3)$$

System Architecture

The receiver is structured modularly with 7 sub-blocks orchestrated by the main top-level wrapper `UART_RX`:

1. `uart_rx_fsm`: Finite State Machine controlling state transitions, sampling enables, check enables, and error clearing.
2. `data_sampling`: Samples `RX_IN` at center ticks and computes the majority vote.
3. `edge_bit_counter`: Tracks edge ticks within a bit period and bit counts within a frame.
4. `deserializer`: Converts serial bits into 8-bit parallel `P_DATA` LSB-first.
5. `strt_check`: Verifies that the start bit remains 0 at sample time.
6. `parity_check`: Calculates expected parity and checks against the received parity bit.
7. `stp_check`: Verifies that the stop bit equals 1.

RTL Source Code

Top Module — `UART_RX.v`

```

1 module UART_RX (
2     input wire      CLK,
3     input wire      RST,
4     input wire      PAR_TYP,
5     input wire      PAR_EN,
6     input wire [5:0] Prescale,
7     input wire      RX_IN,
8     output wire [7:0] P_DATA,
9     output wire      data_valid,
10    output wire      Parity_Error,
11    output wire      Stop_Error

```

```

12 );
13 // Internal Wires matching System Block Diagram
14 wire    dat_samp_en;
15 wire    enable;
16 wire    bit_cnt_en;
17 wire    deser_en;
18 wire    strt_chk_en;
19 wire    strt_glitch;
20 wire    par_chk_en;
21 wire    par_err;
22 wire    stp_chk_en;
23 wire    stp_err;
24 wire    sampled_bit;
25 wire    clear_err;          // Clears par_err/stp_err after CHK_ERR state
26 wire [5:0] edge_cnt;
27 wire [3:0] bit_cnt;
28
29 // Top-Level Error Outputs
30 assign Parity_Error = par_err;
31 assign Stop_Error   = stp_err;
32
33 // 1. FSM Block
34 uart_rx_fsm u_fsm (
35     .CLK(CLK),
36     .RST(RST),
37     .RX_IN(RX_IN),
38     .PAR_EN(PAR_EN),
39     .Prescale(Prescale),
40     .edge_cnt(edge_cnt),
41     .bit_cnt(bit_cnt),
42     .par_err(par_err),
43     .strt_glitch(strt_glitch),
44     .stp_err(stp_err),
45     .dat_samp_en(dat_samp_en),
46     .enable(enable),
47     .bit_cnt_en(bit_cnt_en),
48     .par_chk_en(par_chk_en),
49     .strt_chk_en(strt_chk_en),
50     .stp_chk_en(stp_chk_en),
51     .deser_en(deser_en),
52     .data_valid(data_valid),
53     .clear_err(clear_err)
54 );
55
56 // 2. Data Sampling Block
57 data_sampling u_data_sampling (
58     .CLK(CLK),
59     .RST(RST),
60     .RX_IN(RX_IN),
61     .Prescale(Prescale),
62     .dat_samp_en(dat_samp_en),
63     .edge_cnt(edge_cnt),
64     .sampled_bit(sampled_bit)
65 );
66
67 // 3. Edge and Bit Counter Block
68 edge_bit_counter u_edge_bit_counter (
69     .CLK(CLK),
70     .RST(RST),
71     .enable(enable),
72     .bit_cnt_en(bit_cnt_en),
73     .Prescale(Prescale),
74     .edge_cnt(edge_cnt),
75     .bit_cnt(bit_cnt)
76 );
77
78 // 4. Deserializer Block
79 deserializer u_deserializer (
80     .CLK(CLK),
81     .RST(RST),
82     .deser_en(deser_en),
83     .Prescale(Prescale),
84     .edge_cnt(edge_cnt),

```

```

85     .sampled_bit(sampled_bit),
86     .P_DATA(P_DATA)
87 );
88
89 // 5. Start Check Block
90 strt_check u_strt_check (
91     .CLK(CLK),
92     .RST(RST),
93     .strt_chk_en(strt_chk_en),
94     .sampled_bit(sampled_bit),
95     .strt_glitch(strt_glitch)
96 );
97
98 // 6. Parity Check Block
99 parity_check u_parity_check (
100     .CLK(CLK),
101     .RST(RST),
102     .par_chk_en(par_chk_en),
103     .clear_err(clear_err),
104     .PAR_TYP(PAR_TYP),
105     .sampled_bit(sampled_bit),
106     .P_DATA(P_DATA),
107     .par_err(par_err)
108 );
109
110 // 7. Stop Check Block
111 stp_check u_stp_check (
112     .CLK(CLK),
113     .RST(RST),
114     .stp_chk_en(stp_chk_en),
115     .clear_err(clear_err),
116     .sampled_bit(sampled_bit),
117     .stp_err(stp_err)
118 );
119
120 endmodule

```

Listing 1: UART_RX.v

FSM Controller — uart_rx_fsm.v

```

1 module uart_rx_fsm (
2     input wire      CLK,
3     input wire      RST,
4     input wire      RX_IN,
5     input wire      PAR_EN,
6     input wire [5:0] Prescale,
7     input wire [5:0] edge_cnt,
8     input wire [3:0] bit_cnt,
9     input wire      par_err,
10    input wire      strt_glitch,
11    input wire      stp_err,
12    output reg      dat_samp_en,
13    output reg      enable,
14    output reg      bit_cnt_en,
15    output reg      par_chk_en,
16    output reg      strt_chk_en,
17    output reg      stp_chk_en,
18    output reg      deser_en,
19    output reg      data_valid,
20    output reg      clear_err    // Clears par_err/stp_err after CHK_ERR
21 );
22 // Gray State Encoding
23 localparam IDLE    = 3'b000;
24 localparam START   = 3'b001;
25 localparam DATA    = 3'b011;
26 localparam PARITY   = 3'b010;
27 localparam STOP     = 3'b110;
28 localparam CHK_ERR  = 3'b111;
29
30 reg [2:0] current_state, next_state;

```

```

31
32     wire edge_done;
33     wire bit_done;
34
35     assign edge_done = enable && (edge_cnt == (Prescale - 6'd1));
36     assign bit_done  = (bit_cnt == 4'd7) && edge_done;
37
38     // State Registering
39     always @(posedge CLK or negedge RST) begin
40         if (!RST) begin
41             current_state <= IDLE;
42         end else begin
43             current_state <= next_state;
44         end
45     end
46
47     // Next State & Output Logic
48     always @(*) begin
49         // Default Assignments (Prevents Latches)
50         next_state = current_state;
51         enable     = 1'b0;
52         dat_samp_en = 1'b0;
53         deser_en   = 1'b0;
54         bit_cnt_en = 1'b0;
55         strt_chk_en = 1'b0;
56         par_chk_en = 1'b0;
57         stp_chk_en = 1'b0;
58         data_valid = 1'b0;
59         clear_err  = 1'b0;
60
61         case (current_state)
62             IDLE: begin
63                 if (!RX_IN) begin
64                     next_state = START;
65                 end
66             end
67
68             START: begin
69                 enable     = 1'b1;
70                 dat_samp_en = 1'b1;
71
72                 if (edge_done) begin
73                     strt_chk_en = 1'b1;
74                     if (!strt_glitch) begin
75                         next_state = DATA;
76                     end else begin
77                         next_state = IDLE;
78                     end
79                 end
80             end
81
82             DATA: begin
83                 enable     = 1'b1;
84                 dat_samp_en = 1'b1;
85                 deser_en   = 1'b1;
86                 bit_cnt_en = 1'b1;
87                 if (bit_done) begin
88                     if (PAR_EN) begin
89                         next_state = PARITY;
90                     end else begin
91                         next_state = STOP;
92                     end
93                 end
94             end
95
96             PARITY: begin
97                 enable     = 1'b1;
98                 dat_samp_en = 1'b1;
99                 if (edge_done) begin
100                     par_chk_en = 1'b1;
101                     next_state = STOP;
102                 end
103             end

```

```

104
105         STOP: begin
106             enable      = 1'b1;
107             dat_samp_en = 1'b1;
108             if (edge_done) begin
109                 stp_chk_en = 1'b1;
110                 next_state = CHK_ERR;
111             end
112         end
113
114         CHK_ERR: begin
115             if (!par_err && !stp_err) begin
116                 data_valid = 1'b1;
117             end
118             clear_err = 1'b1; // Always clear errors after checking
119
120             // Back-to-Back: start next frame immediately if valid
121             if (!RX_IN && !par_err && !stp_err) begin
122                 next_state = START;
123             end else begin
124                 next_state = IDLE;
125             end
126         end
127
128         default: begin
129             next_state = IDLE;
130         end
131     endcase
132 end
133 endmodule

```

Listing 2: uart_rx_fsm.v

Data Sampling Module — data_sampling.v

```

1 module data_sampling (
2     input wire    CLK,
3     input wire    RST,
4     input wire    RX_IN,
5     input wire [5:0] Prescale,
6     input wire    dat_samp_en,
7     input wire [5:0] edge_cnt,
8     output reg    sampled_bit
9 );
10 // 3-sample majority voting at the middle of the bit period
11 // Samples taken at: (Prescale/2 - 1), (Prescale/2), (Prescale/2 + 1)
12 // Supported Prescale values: 8, 16, 32
13 // For Prescale=8: sample ticks = 3, 4, 5
14 // For Prescale=16: sample ticks = 7, 8, 9
15 // For Prescale=32: sample ticks = 15, 16, 17
16
17 reg [2:0] samples;
18 wire [5:0] half_ticks;
19
20 assign half_ticks = Prescale >> 1;
21
22 always @(posedge CLK or negedge RST) begin
23     if (!RST) begin
24         samples <= 3'b111;
25         sampled_bit <= 1'b1;
26     end else if (dat_samp_en) begin
27         // Sample 0 at (Prescale/2 - 1)
28         if (edge_cnt == (half_ticks - 6'd1))
29             samples[0] <= RX_IN;
30
31         // Sample 1 at (Prescale/2)
32         if (edge_cnt == half_ticks)
33             samples[1] <= RX_IN;
34
35         // Sample 2 at (Prescale/2 + 1) → majority vote registered here
36         if (edge_cnt == (half_ticks + 6'd1)) begin

```

```

37         samples[2]  <= RX_IN;
38         sampled_bit <= (samples[0] & samples[1]) |
39                        (samples[0] & RX_IN)      |
40                        (samples[1] & RX_IN);
41     end
42 end else begin
43     samples      <= 3'b111;
44     sampled_bit <= 1'b1;
45 end
46 end
47 endmodule

```

Listing 3: data_sampling.v

Edge and Bit Counter — edge_bit_counter.v

```

1 module edge_bit_counter (
2     input  wire    CLK,
3     input  wire    RST,
4     input  wire    enable,
5     input  wire    bit_cnt_en,
6     input  wire [5:0] Prescale,
7     output reg [5:0] edge_cnt,
8     output reg [3:0] bit_cnt
9 );
10 always @(posedge CLK or negedge RST) begin
11     if (!RST) begin
12         edge_cnt <= 6'd0;
13         bit_cnt  <= 4'd0;
14     end else if (enable) begin
15         if (edge_cnt == (Prescale - 6'd1)) begin
16             edge_cnt <= 6'd0;
17             if (bit_cnt_en) begin
18                 if (bit_cnt == 4'd7) begin
19                     bit_cnt <= 4'd0;
20                 end else begin
21                     bit_cnt <= bit_cnt + 4'd1;
22                 end
23             end else begin
24                 bit_cnt <= 4'd0;
25             end
26         end else begin
27             edge_cnt <= edge_cnt + 6'd1;
28         end
29     end else begin
30         edge_cnt <= 6'd0;
31         bit_cnt  <= 4'd0;
32     end
33 end
34 endmodule

```

Listing 4: edge_bit_counter.v

Deserializer Module — deserializer.v

```

1 module deserializer (
2     input  wire    CLK,
3     input  wire    RST,
4     input  wire    deser_en,
5     input  wire [5:0] Prescale,
6     input  wire [5:0] edge_cnt,
7     input  wire    sampled_bit,
8     output reg [7:0] P_DATA
9 );
10 // UART sends LSB first (b0 → b7)
11 // Shift sampled_bit in from the RIGHT (LSB) on each bit-done edge
12 always @(posedge CLK or negedge RST) begin
13     if (!RST) begin

```

```

14     P_DATA <= 8'd0;
15     end else if (deser_en && (edge_cnt == (Prescale - 6'd1))) begin
16         P_DATA <= {sampled_bit, P_DATA[7:1]}; // MSB shift-in, fills b0 first -> LSB-
17         ↪ first
18     end
19 endmodule

```

Listing 5: deserializer.v

Start Bit Checker — strt_check.v

```

1 module strt_check (
2     input wire CLK,
3     input wire RST,
4     input wire strt_chk_en,
5     input wire sampled_bit,
6     output reg strt_glitch
7 );
8     always @(posedge CLK or negedge RST) begin
9         if (!RST) begin
10             strt_glitch <= 1'b0;
11         end else if (strt_chk_en) begin
12             strt_glitch <= (sampled_bit != 1'b0);
13         end else begin
14             strt_glitch <= 1'b0;
15         end
16     end
17 endmodule

```

Listing 6: strt_check.v

Parity Checker — parity_check.v

```

1 module parity_check (
2     input wire CLK,
3     input wire RST,
4     input wire par_chk_en,
5     input wire clear_err, // FSM clears this after CHK_ERR
6     input wire PAR_TYP,
7     input wire sampled_bit,
8     input wire [7:0] P_DATA,
9     output reg par_err
10 );
11     wire expected_parity;
12     assign expected_parity = (PAR_TYP == 1'b0) ? (~P_DATA) : ~(~P_DATA);
13
14     always @(posedge CLK or negedge RST) begin
15         if (!RST) begin
16             par_err <= 1'b0;
17         end else if (clear_err) begin
18             par_err <= 1'b0; // Clear after CHK_ERR - ready for next frame
19         end else if (par_chk_en) begin
20             par_err <= (sampled_bit != expected_parity);
21         end
22     end
23 endmodule

```

Listing 7: parity_check.v

Stop Bit Checker — stp_check.v

```

1 module stp_check (
2     input wire CLK,
3     input wire RST,
4     input wire stp_chk_en,
5     input wire clear_err,      // FSM clears this after CHK_ERR
6     input wire sampled_bit,
7     output reg stp_err
8 );
9     always @(posedge CLK or negedge RST) begin
10         if (!RST) begin
11             stp_err <= 1'b0;
12         end else if (clear_err) begin
13             stp_err <= 1'b0;      // Clear after CHK_ERR - ready for next frame
14         end else if (stp_chk_en) begin
15             stp_err <= (sampled_bit != 1'b1);
16         end
17     end
18 endmodule

```

Listing 8: stp_check.v

SpyGlass Lint Audit

The SpyGlass project file (`lint.prj`) and report (`moresimple.rpt`) inside `UART_SPYGLASS` were audited:

`moresimple.rpt` Summary

Table 3: SpyGlass Report Overview

Parameter	Value
Total Generated Messages	3
Number of Errors	0
Number of Warnings	1 (STARC05-2.11.3.1)
Design Read Messages	2 (Info)

Rule Analysis

- **DetectTopDesignUnits (Info):** Identified `UART_RX` correctly as top design unit.
- **ElabSummary (Info):** Elaboration completed successfully with zero missing hierarchy components.
- **STARC05-2.11.3.1 (Warning on `edge_bit_counter.v`):** Flagged because edge and bit counter logic are implemented in a single sequential block. This is a design choice that maintains clean counter synchrony without creating any latches or functional risks.

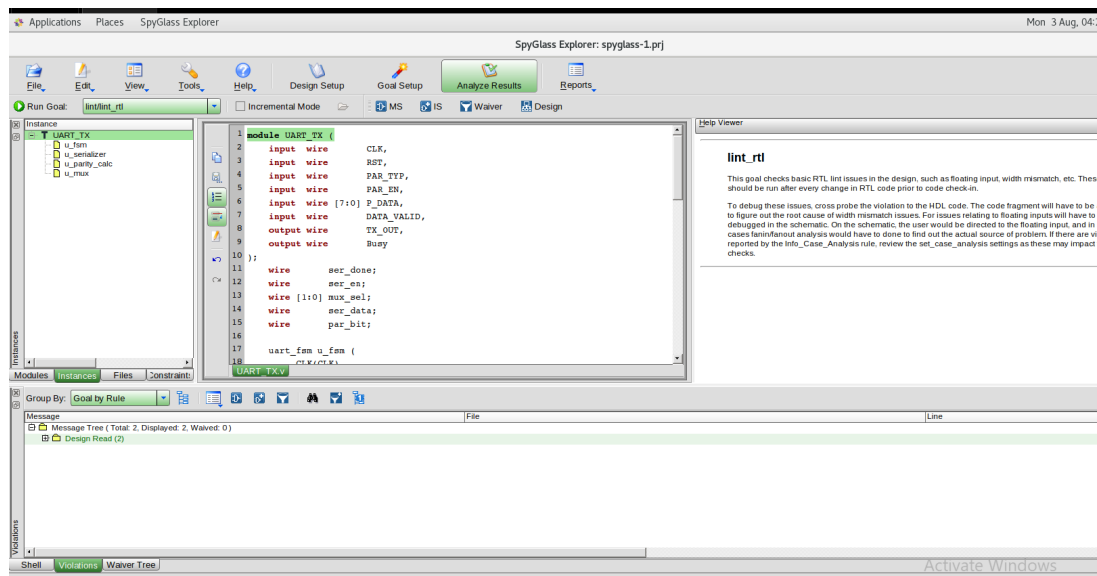


Figure 1: SpyGlass Lint Summary Result

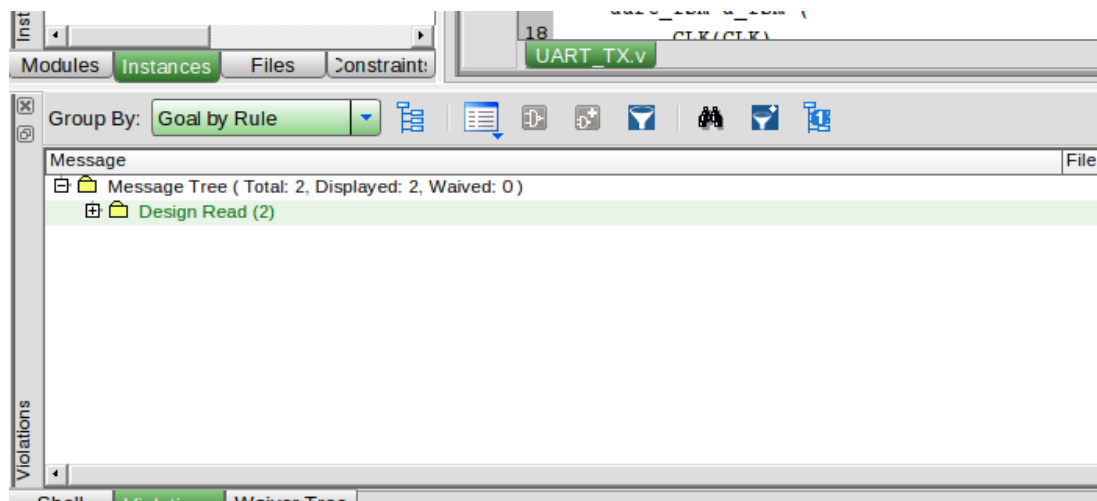
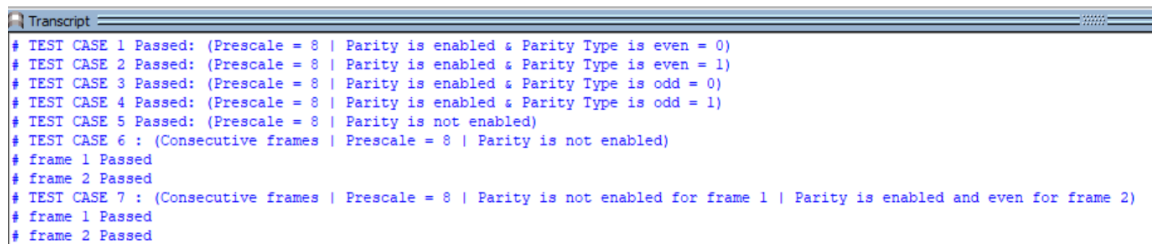


Figure 2: SpyGlass Design Read Report

Simulation & Verification Results

The design was verified using ModelSim Intel FPGA Edition v10.5b with a self-checking testbench (UART_RX_tb.v).

ModelSim Transcripts

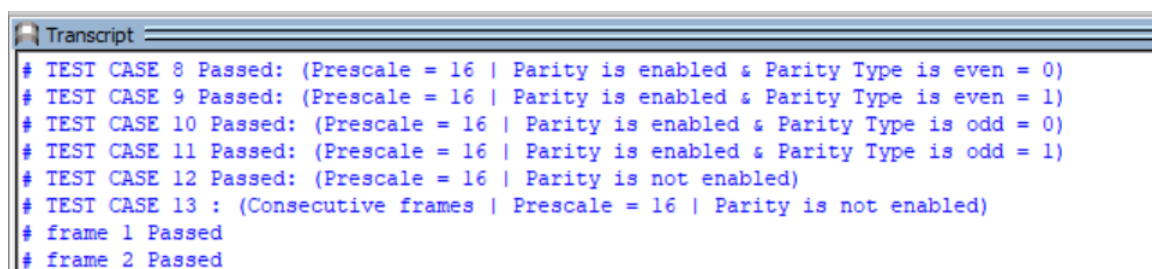


```

# TEST CASE 1 Passed: (Prescale = 8 | Parity is enabled & Parity Type is even = 0)
# TEST CASE 2 Passed: (Prescale = 8 | Parity is enabled & Parity Type is even = 1)
# TEST CASE 3 Passed: (Prescale = 8 | Parity is enabled & Parity Type is odd = 0)
# TEST CASE 4 Passed: (Prescale = 8 | Parity is enabled & Parity Type is odd = 1)
# TEST CASE 5 Passed: (Prescale = 8 | Parity is not enabled)
# TEST CASE 6 : (Consecutive frames | Prescale = 8 | Parity is not enabled)
# frame 1 Passed
# frame 2 Passed
# TEST CASE 7 : (Consecutive frames | Prescale = 8 | Parity is not enabled for frame 1 | Parity is enabled and even for frame 2)
# frame 1 Passed
# frame 2 Passed

```

Figure 3: ModelSim Verification Log — Prescale = 8 (Even/Odd/No Parity & Consecutive Frames)

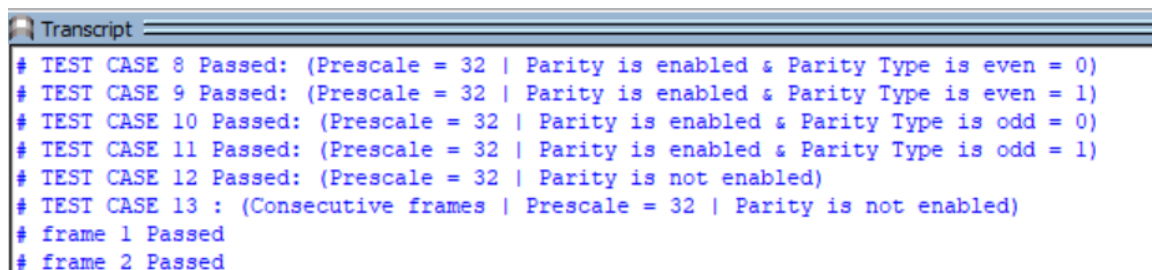


```

# TEST CASE 8 Passed: (Prescale = 16 | Parity is enabled & Parity Type is even = 0)
# TEST CASE 9 Passed: (Prescale = 16 | Parity is enabled & Parity Type is even = 1)
# TEST CASE 10 Passed: (Prescale = 16 | Parity is enabled & Parity Type is odd = 0)
# TEST CASE 11 Passed: (Prescale = 16 | Parity is enabled & Parity Type is odd = 1)
# TEST CASE 12 Passed: (Prescale = 16 | Parity is not enabled)
# TEST CASE 13 : (Consecutive frames | Prescale = 16 | Parity is not enabled)
# frame 1 Passed
# frame 2 Passed

```

Figure 4: ModelSim Verification Log — Prescale = 16 (All Frame Types & Consecutive Frames)



```

# TEST CASE 8 Passed: (Prescale = 32 | Parity is enabled & Parity Type is even = 0)
# TEST CASE 9 Passed: (Prescale = 32 | Parity is enabled & Parity Type is even = 1)
# TEST CASE 10 Passed: (Prescale = 32 | Parity is enabled & Parity Type is odd = 0)
# TEST CASE 11 Passed: (Prescale = 32 | Parity is enabled & Parity Type is odd = 1)
# TEST CASE 12 Passed: (Prescale = 32 | Parity is not enabled)
# TEST CASE 13 : (Consecutive frames | Prescale = 32 | Parity is not enabled)
# frame 1 Passed
# frame 2 Passed

```

Figure 5: ModelSim Verification Log — Prescale = 32 (All Frame Types & Consecutive Frames)

